

MongoDB & Ontologies OWL/RDF

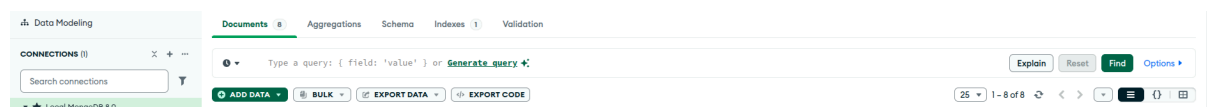
[Brief_mongoDB_1_partie1.pdf](#)

1.2 — Observer

Q3 — Combien de documents ?

Combien de documents contient la collection ?

```
greg@sadhill:~/Documents/Sinplon/streamvault-semantic-mongodb$ mongosh --eval "db.getSiblingDB('bibliotheque').livres.countDocuments()"
8
greg@sadhill:~/Documents/Sinplon/streamvault-semantic-mongodb$
```



Q4 — Le premier document

Affiche le tout premier document. Quel est son titre ?

```
greg@sadhill:~/Documents/Sinplon/streamvault-semantic-mongodb$ mongosh --eval "db.getSiblingDB('bibliotheque').livres.findOne()"
{
  "_id": ObjectId("6aa1793246e1826f88992bca"),
  "titre": "Le Petit Prince",
  "auteur": "Antoine de Saint-Exupéry",
  "annee": 1943
}
```



Q5 — Le deuxième document

Affiche le deuxième document (pas le premier). Qui en est l'auteur ?

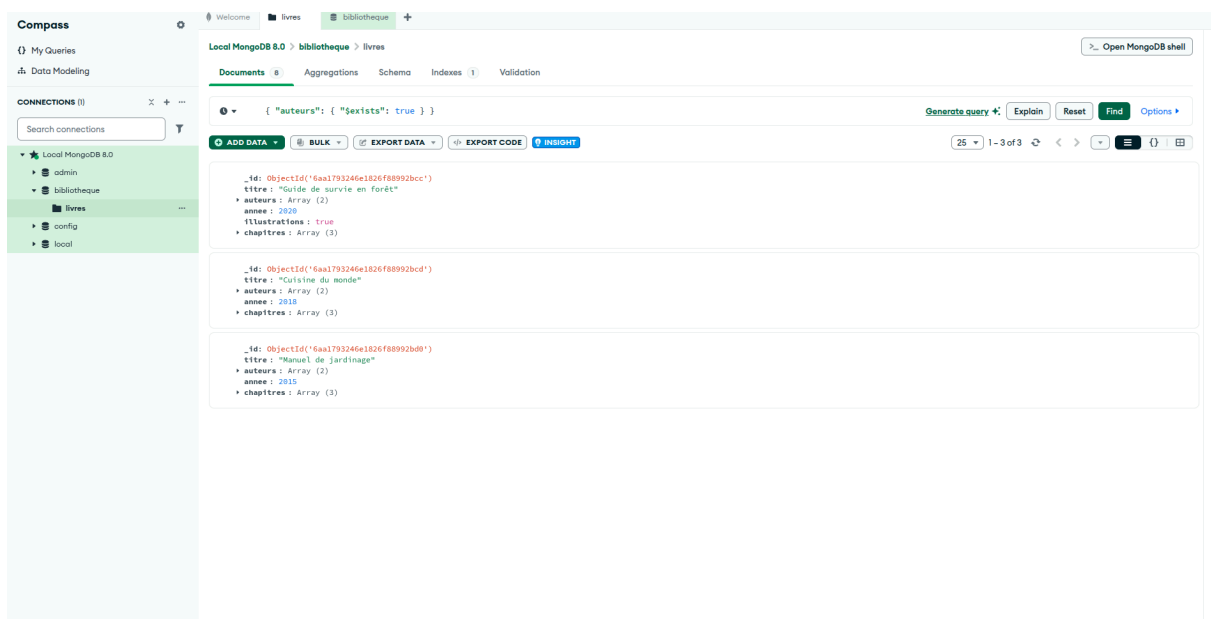
```
greg@sadhill:~/Documents/Sinplon/streamvault-semantic-mongodb$ mongosh --eval "db.getSiblingDB('bibliotheque').livres.find().skip(1).limit(1)"
{
  "_id": ObjectId("6aa1793246e1826f88992bcb"),
  "titre": "1984",
  "auteur": "George Orwell",
  "annee": 1949
}
```



Q6 — Un document précis

Trouve le document dont le titre est celui de ton livre à auteurs multiples. A-t-il bien un champ auteurs au pluriel ?

```
greg@sadhill: ~/Documents/Sinplon/streamvault-semantic-nongodh$ mongosh --eval "db.getSiblingDB('bibliotheque').livres.find({ auteurs: { $exists: true } })"
[
  {
    _id: ObjectId('6aa1793246e1826f88992bcc'),
    titre: 'Guide de survie en forêt',
    auteurs: [ 'J. Dupont', 'M. Martin' ],
    annee: 2020,
    illustrations: true,
    chapitres: [ 'Orientation', 'Feu', 'Abri' ]
  },
  {
    _id: ObjectId('6aa1793246e1826f88992bcd'),
    titre: 'Cuisine du monde',
    auteurs: [ 'A. Bernard', 'C. Petit' ],
    annee: 2018,
    chapitres: [ 'Entrées', 'Plats', 'Desserts' ]
  },
  {
    _id: ObjectId('6aa1793246e1826f88992bd0'),
    titre: 'Manuel de jardinage',
    auteurs: [ 'S. Roux', 'P. Blanc' ],
    annee: 2015,
    chapitres: [ 'Potager', 'Fleurs', 'Arbres' ]
  }
]
```



Q7 — Chapitres

Ce même document a-t-il un champ chapitres ? Liste-les.

```
greg@sadhill: ~/Documents/Sinplon/streamvault-semantic-nongodh$ mongosh --eval "db.getSiblingDB('bibliotheque').livres.findOne({ titre: 'Guide de survie en forêt' }, { chapitres: 1, _id: 0 })"
{ chapitres: [ 'Orientation', 'Feu', 'Abri' ] }
greg@sadhill: ~/Documents/Sinplon/streamvault-semantic-nongodh$
```

The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel lists 'Local MongoDB 6.0' with a tree view showing 'admin', 'bibliotheque', 'livres', 'config', and 'local'. The 'Documents' tab is active, displaying a query: `{ "titre": "Guide de survie en forêt" }`. The results show a single document with the following structure:

```

{
  "_id": ObjectId('6aa1793246e1826f88992bcc'),
  "titre": "Guide de survie en forêt",
  "auteurs": Array(2),
  "annee": 2020,
  "illustrations": true,
  "chapters": Array(3)
}

```

The 'chapters' array contains: 'Orientation', 'Feu', and 'Abri'.

Q8 — Le plus ancien

Trouve le document le plus ancien de la collection.

```

greg@sadhill:~/Documents/Simplon/streamvault-semantic-mongodb$ mongosh --eval "db.getSiblingDB('bibliotheque').livres.find().sort({ annee: 1 }).limit(1)"
{
  "_id": ObjectId('6aa1793246e1826f88992bcf'),
  "titre": "L'étranger",
  "auteurs": "Albert Camus",
  "annee": 1942
}
greg@sadhill:~/Documents/Simplon/streamvault-semantic-mongodb$

```

The screenshot shows the MongoDB Compass interface. The 'Sort' field is set to `{ "annee": -1 }`. The results show two documents, sorted by year (oldest first):

```

{
  "_id": ObjectId('6aa1793246e1826f88992bcf'),
  "titre": "L'étranger",
  "auteurs": "Albert Camus",
  "annee": 1942
}
{
  "_id": ObjectId('6aa1793246e1826f88992bca'),
  "titre": "Le Petit Prince",
  "auteurs": "Antoine de Saint-Exupéry",
  "annee": 1943
}

```

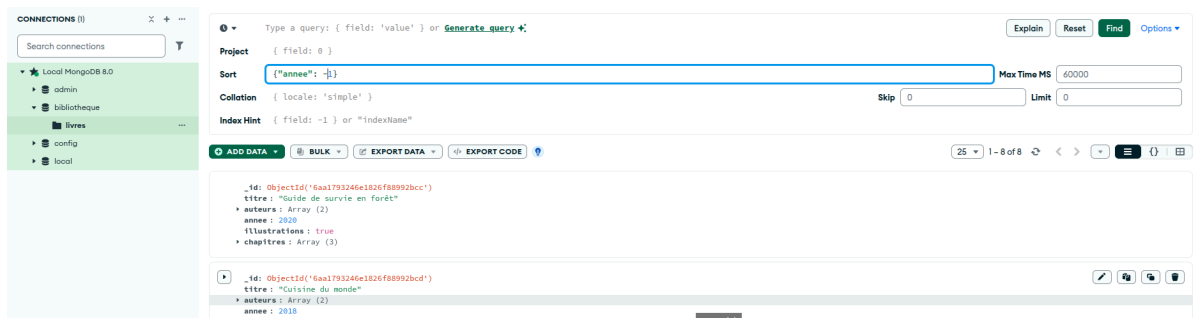
Q9 — Le plus récent

Trouve le document le plus récent.

```

greg@sadhill:~/Documents/Simplon/streamvault-semantic-mongodb$ mongosh --eval "db.getSiblingDB('bibliotheque').livres.find().sort({ annee: -1 }).limit(1)"
{
  "_id": ObjectId('6aa1793246e1826f88992bcc'),
  "titre": "Guide de survie en forêt",
  "auteurs": [ "J. Dupont", "M. Martin" ],
  "annee": 2020,
  "illustrations": true,
  "chapters": [ "Orientation", "Feu", "Abri" ]
}
greg@sadhill:~/Documents/Simplon/streamvault-semantic-mongodb$

```



Q10 — Champ unique

Un seul document a un champ qui n'existe sur aucun autre. Lequel, et lequel est ce champ ?

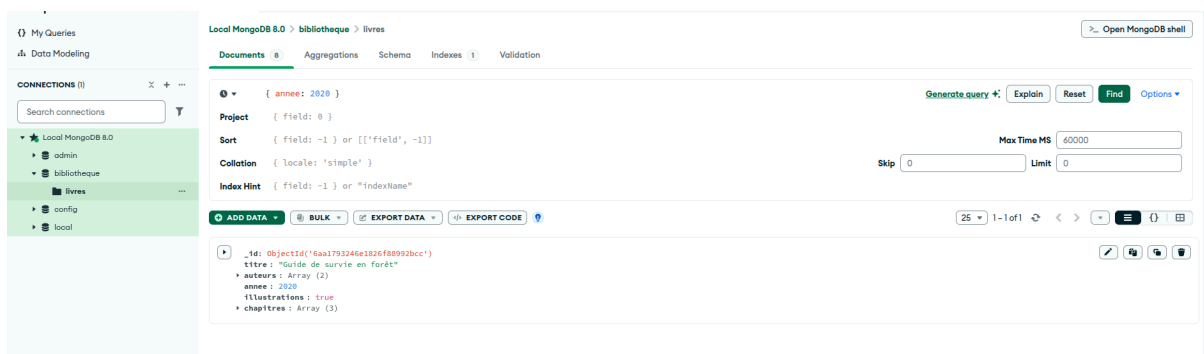
```
greg@sadhill:~/Documents/Sinplon/streamvault-semantic-mongodb$ mongosh --eval "db.getSiblingDB('bibliotheque').livres.aggregate([ { $match: { illustrations: { $exists: true } } }, { $project: { titre: 1, illustrations: 1, id: 0 } } ])"
[ { titre: 'Guide de survie en forêt', illustrations: true } ]
greg@sadhill:~/Documents/Sinplon/streamvault-semantic-mongodb$
```

1.3 — Filtrer (barre de filtre Compass)

Q11 — Filtre sur une année précise

Trouve le livre publié une année précise de ton choix.

```
greg@sadhill:~/Documents/Sinplon/streamvault-semantic-mongodb$ mongosh --eval "db.getSiblingDB('bibliotheque').livres.find({ annee: 2020 })"
[ { _id: ObjectId('6aa1793246e1826f88992bcc'),
  titre: 'Guide de survie en forêt',
  auteurs: [ 'J. Dupont', 'M. Martin' ],
  annee: 2020,
  illustrations: true,
  chapitres: [ 'Orientation', 'Feu', 'Abri' ]
}]
greg@sadhill:~/Documents/Sinplon/streamvault-semantic-mongodb$
```



Q12 — Filtre ">"

Combien de livres ont été publiés après 2000 ?

```
greg@sadhill:~/Documents/Simplon/streamvault-semantic-mongod$ mongosh --eval "db.getSiblingDB('bibliotheque').livres.find({ annee: { $gt: 2000 } }).count()"
3
greg@sadhill:~/Documents/Simplon/streamvault-semantic-mongod$
```

Local MongoDB 8.0 > bibliotheque > livres

Documents Aggregations Schema Indexes Validation

Query: { "annee": { "\$gt": 2000 } }

Project: { field: 0 }

Sort: { field: -1 } or [{"field": -1}]

Collation: { locale: 'simple' }

Index Hint: { field: -1 } or "indexName"

Max Time MS: 60000

Skip: 0 Limit: 0

25 1-3 of 3

Document
<pre>{ "_id": ObjectId("6aa1793246e1826f88992bcc"), "titre": "Guide de survie en forêt", "auteurs": Array (2), "annee": 2020, "illustrations": true, "chapters": Array (3) }</pre>
<pre>{ "_id": ObjectId("6aa1793246e1826f88992bcd"), "titre": "Cuisine du monde", "auteurs": Array (2), "annee": 2018, "chapters": Array (3) }</pre>
<pre>{ "_id": ObjectId("6aa1793246e1826f88992bd0"), "titre": "Manuel de jardinage", "auteurs": Array (2), "annee": 2015, "chapters": Array (3) }</pre>

Q13 — Filtre "<"

Combien de livres ont été publiés avant 1950 ?

```
greg@sadhill:~/Documents/Simplon/streamvault-semantic-mongod$ mongosh --eval "db.getSiblingDB('bibliotheque').livres.find({ annee: { $lt: 1950 } }).count()"
3
greg@sadhill:~/Documents/Simplon/streamvault-semantic-mongod$
```

Local MongoDB 8.0 > bibliotheque > livres

Documents Aggregations Schema Indexes Validation

Query: { "annee": { "\$lt": 1950 } }

Project: { field: 0 }

Sort: { field: -1 } or [{"field": -1}]

Collation: { locale: 'simple' }

Index Hint: { field: -1 } or "indexName"

Max Time MS: 60000

Skip: 0 Limit: 0

25 1-3 of 3

Document
<pre>{ "_id": ObjectId("6aa1793246e1826f88992bca"), "titre": "Le Petit Prince", "auteur": "Antoine de Saint-Exupéry", "annee": 1943 }</pre>
<pre>{ "_id": ObjectId("6aa1793246e1826f88992bcb"), "titre": "1984", "auteur": "George Orwell", "annee": 1949 }</pre>
<pre>{ "_id": ObjectId("6aa1793246e1826f88992bcf"), "titre": "L'Étranger", "auteur": "Albert Camus", "annee": 1942 }</pre>

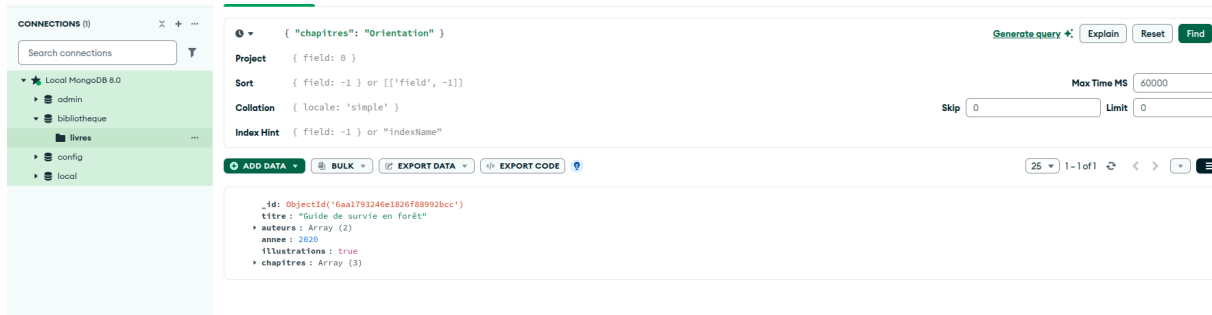
Q14 — Filtre sur un tableau

Trouve le livre qui contient un chapitre précis.

```

greg@sadhill:~/Documents/Simplon/streamvault-semantic-mongodb$ mongosh --eval "db.getSiblingDB('bibliotheque').livres.find({ chapitres: 'Orientation' })"
[
  {
    _id: ObjectId('6aa1793246e1826f88992bcc'),
    titre: 'Guide de survie en forêt',
    auteurs: [ 'J. Dupont', 'M. Martin' ],
    annee: 2020,
    illustrations: true,
    chapitres: [ 'Orientation', 'Feu', 'Abri' ]
  }
]
greg@sadhill:~/Documents/Simplon/streamvault-semantic-mongodb$

```



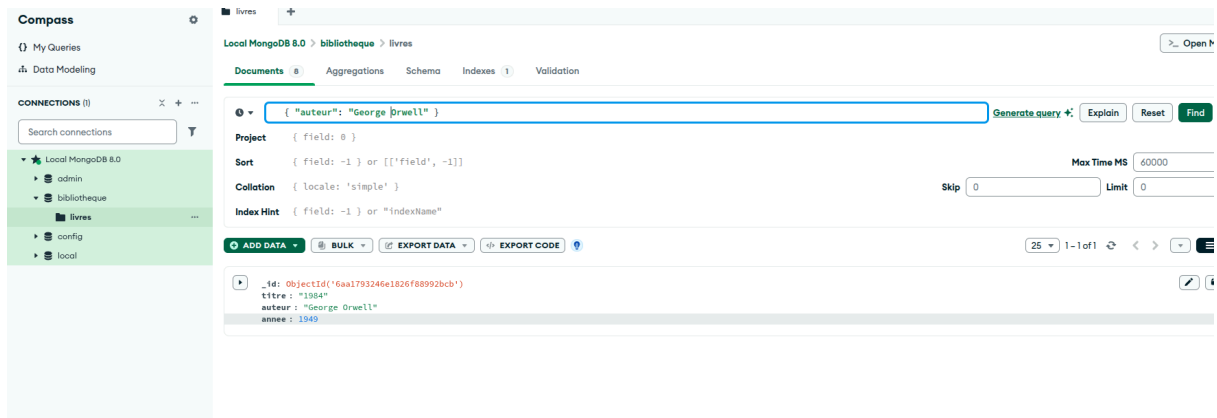
Q15 — Filtre sur l'auteur

Trouve un livre à partir du nom exact de son auteur.

```

greg@sadhill:~/Documents/Simplon/streamvault-semantic-mongodb$ mongosh --eval "db.getSiblingDB('bibliotheque').livres.find({ auteur: 'George Orwell' })"
[
  {
    _id: ObjectId('6aa1793246e1826f88992bcb'),
    titre: '1984',
    auteur: 'George Orwell',
    annee: 1949
  }
]
greg@sadhill:~/Documents/Simplon/streamvault-semantic-mongodb$

```



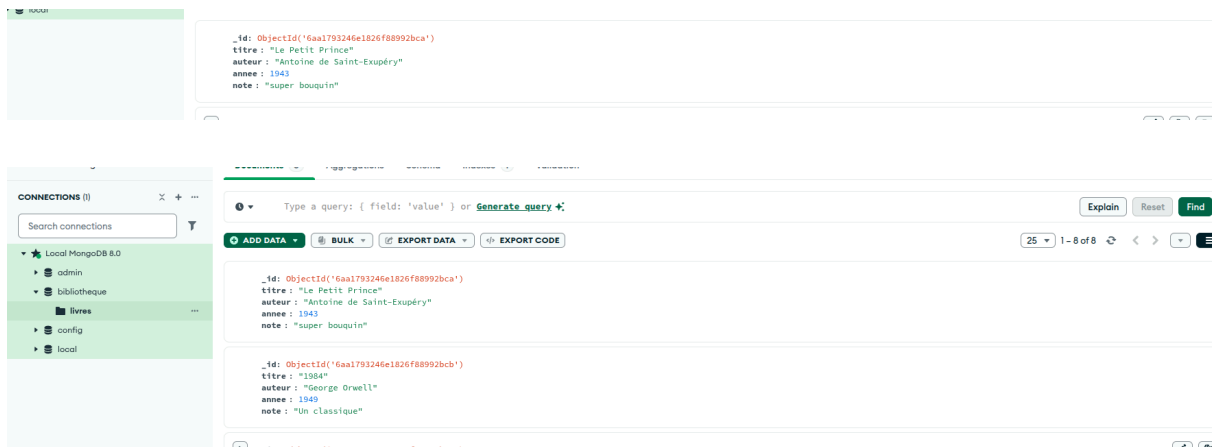
1.4 — Modifier (toujours en visuel)

Q16 — Ajouter un champ

Ouvre un document et ajoute-lui un champ note avec une valeur de ton choix.

Q17 — Vérifier l'ajout

Rouvre ce document pour confirmer que le champ est bien là.



Q18 — Ajouter un document

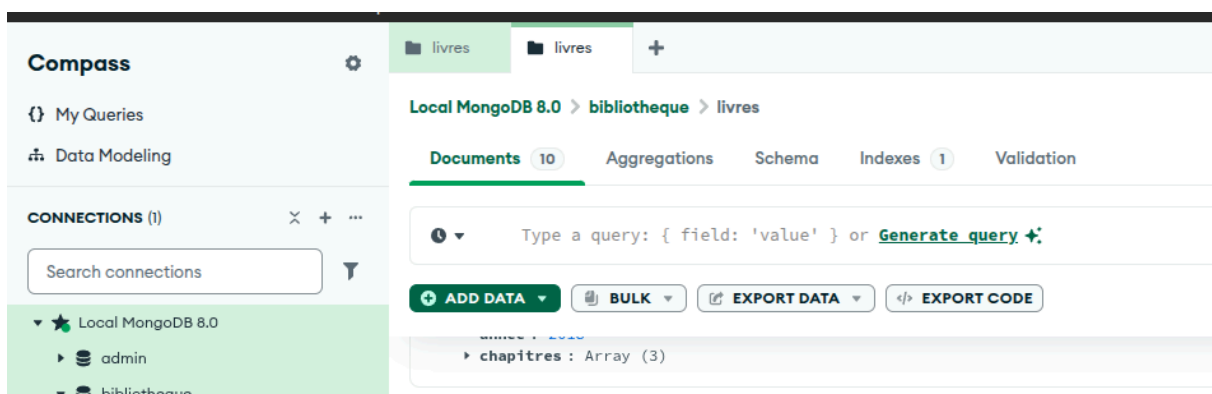
Insère un nouveau document avec un titre et une année de ton choix. Creation 2livres

Q19 — Vérifier le compteur

Le nombre total de documents a-t-il bien augmenté de 2 ?



```
greg@sadhill:~/Documents/Sinlon/streamvault-semantic-mongodb$ mongosh --eval "db.getSiblingDB('bibliotheque').livres.insertOne({ titre: 'Les Sentinelles de l'Algorithme', auteur: 'E. Turin g', annee: 2025, theme: 'IA & Sci-Fi' })"
{ acknowledged: true,
  insertedId: ObjectId('6aa18edf04522b9c49076fab')
}
greg@sadhill:~/Documents/Sinlon/streamvault-semantic-mongodb$
```

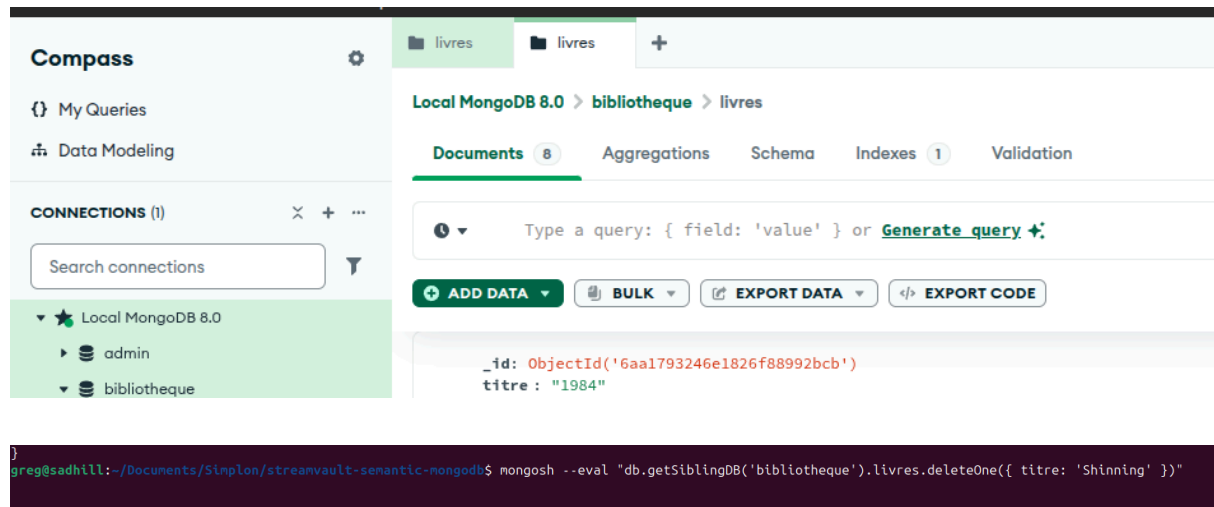


Q20 — Supprimer

Supprime ce document ajouté. Le compteur est-il revenu à sa valeur initiale ?

💡 ✅ VALIDATION NIVEAU 1 : les 20 questions ont une réponse correcte, obtenue en manipulant

l'interface Compass sans écrire de requête complexe.



Partie 2 - Construire manuellement une ontologie de films (OWL/RDF pur, sans base de données)

[brief1_partie2_ontologie_owl_rdf.pdf](#)

Brief 1 — Partie 2

Construire manuellement une ontologie de films (OWL/RDF pur, sans base de données)

WebProtégé (construction) → WebVOWL (visualisation)

Contexte

Ce brief n'utilise aucune base de données et aucun code. L'objectif est de comprendre, uniquement en manipulant, ce qu'est une ontologie : des classes, des relations nommées entre elles, des individus concrets, et un raisonneur capable d'en déduire de nouveaux faits. Le domaine choisi (le cinéma) est indépendant du reste de votre parcours aucune donnée réelle, aucun MongoDB, aucun script ici.

RDF et OWL, deux outils, deux moments

Ce brief utilise deux outils, chacun pour une raison précise :

Outil	Ce que vous y faites	Ce que ça illustre
WebProtégé	Construire les classes, les propriétés, les individus et les relations entre eux https://webprotege.stanford.edu/	RDF — le graphe de triplets (sujet-prédicat-objet) qui contient les vraies données
WebVOWL	Visualiser le résultat sous forme de graphe https://service.tib.eu/webvowl/	OWL — le schéma conceptuel (les classes et leurs relations possibles), lu à un niveau plus abstrait

💡 En résumé : WebProtégé pour construire les relations concrètes (RDF), WebVOWL pour voir le schéma qui en résulte (OWL). Les deux sont les deux faces de la même ontologie.

1 — Les classes et sous-classes

À créer dans l'onglet "Classes" de WebProtégé.

Classe	Sous-classe de
Personne	owl:Thing
Acteur	Personne
Realisateur	Personne
Film	owl:Thing
Genre	owl:Thing
Studio	owl:Thing

2 — Les Object Properties (relations entre classes)

Pour chaque ligne : créer la propriété, renseigner Domain et Range, puis la relier à son inverse via le champ "Inverse property".

Propriété	Domain	Range	Inverse	Caractéristique
aJoueDans	Acteur	Film	aPourActeur	—
aRealise	Realisateur	Film	estRealisePar	—
aPourGenre	Film	Genre	estDuGenre	—
estProduitPar	Film	Studio	aProduit	—
estSuiteDe	Film	Film	aPourSuite	Volontairement NON transitive
estFilialeDe	Studio	Studio	aPourFiliale	Transitive

3 — Les Data Properties (attributs simples)

Propriété	Domain	Type	Caractéristique
aPourTitre	Film	string	Functional
aPourAnnee	Film	integer	Functional
aPourNoteIMDB	Film	float	Functional
aPourIdentifiantIMDB	Film	string	Functional
aPourNom	Personne	string	Functional

4 — Les individus à créer

Individu	Classe	Valeurs (Data Properties)
ChristopherNolan	Realisateur	aPourNom : "Christopher Nolan"
LeonardoDiCaprio	Acteur	aPourNom : "Leonardo DiCaprio"
Inception	Film	aPourTitre : "Inception" · aPourAnnee : 2010 · aPourNoteIMDB : 8.8 · aPourIdentifiantIMDB : "tt1375666"
Casablanca	Film	aPourTitre : "Casablanca" · aPourAnnee : 1942
ActionGenre	Genre	—
DrameGenre	Genre	—
WarnerBros	Studio	—
WarnerAnimation	Studio	—
WarnerBrosGroup	Studio	—

5 — Les relations à créer entre individus

Sur chaque individu, section "Property assertions", ajouter :

ChristopherNolan	--aRealise-->	Inception
LeonardoDiCaprio	--aJoueDans-->	Inception
Inception	--aPourGenre-->	ActionGenre
Inception	--estProduitPar-->	WarnerBros
WarnerAnimation	--estFilialeDe-->	WarnerBros
WarnerBros	--estFilialeDe-->	WarnerBrosGroup

La transitivité (estFilialeDe)

💡 ✔ Résultat attendu : le raisonneur doit ajouter, de lui-même, la relation WarnerAnimation --estFilialeDe--> WarnerBrosGroup — sans que vous l'ayez jamais écrite explicitement. C'est la transitivité en action.

6 — Exporter et visualiser

- Dans WebProtégé : menu du projet → Download → format OWL/XML Sur <https://service.tib.eu/webvowl/>
- : "Select ontology file" → charger le fichier téléchargé
- Observer le graphe : les classes en cercles, les propriétés comme flèches nommées entre elles

! ✓ **VALIDATION BRIEF 1, PARTIE 2** : les 6 classes, les 6 Object Properties (avec inverses), les 5 Data Properties, les 9 individus et leurs relations sont créés. Les deux tests du raisonneur donnent le résultat attendu (déduction pour l'un, incohérence pour l'autre). Le graphe WebVOWL est généré et lisible.

